The master theorem

CS3230 · Lecture 4

Demo course material, written for OpenPapr

Recurrences of one shape

- $T(n) = aT(n/b) + f(n)$
- a subproblems, each of size n/b
- f(n): the work to divide and combine

The critical exponent

- Compare f(n) with n^(log_b a)
- n^(log_b a) counts the leaves of the recursion tree

Case 1

- If f(n) = O(n^(log_b a - e)) for some e > 0
- then T(n) = Theta(n^(log_b a))
- The leaves dominate

Case 2

- If f(n) = Theta(n^(log_b a))
- then T(n) = Theta(n^(log_b a) log n)
- Every level costs the same

Case 3

- If f(n) = Omega(n^(log_b a + e)) for some e > 0, and a f(n/b) <= c f(n) for some c < 1
- then T(n) = Theta(f(n))
- The root dominates

The theorem on one slide

- For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a)
- f polynomially smaller: Theta(n^(log_b a))
- f the same: Theta(n^(log_b a) log n)
- f polynomially larger, and regular: Theta(f(n))

Examples

- 2T(n/2) + n: Case 2, Theta(n log n)
- 8T(n/2) + n^2: Case 1, Theta(n^3)
- 2T(n/2) + n^2: Case 3, Theta(n^2)

When it does not apply

- $T(n) = 2T(n/2) + n \log n$ falls in the gap between Cases 2 and 3
- Use the recursion tree or substitution instead

Summary

- Find a, b and f
- Compare f(n) with n^(log_b a)
- Pick the case; check regularity for Case 3